

First-principles pre-calcliner and cooler models

Status	In progress
Plants	Heidelberg Mokra
Created	April 23, 2026 8:10 AM
Last edited	May 20, 2026 11:50 PM
Features	
Author	 Michael Craig
Tags	
Type	Ideas
Project	 Physics-based models

Precalcliner progress summary

Generated a summary of the progress of the research to date with Claude including the thermodynamic principles used, the data sources used to apply said principles and how we are evaluating the resulting model

[summary_jax.html](#)

Claude MOK Preheater Tower — First-Principles Simulator

Branch: `extend-bagterp` · 7 commits on top of `main` · Source tree: `exp/users/michael/physics_simulator/`

0. What is a preheater tower?

The **preheater tower** is the vertical stack of cyclones bolted onto the back end of a modern dry-process cement kiln. Its job is to dry, heat and partially **calcine** the raw meal (a ground mix of limestone, shale and iron oxide) before it ever enters the rotary kiln. It does this by recovering heat from the kiln exhaust gas: the gas rises, the meal falls, and over four stages they counter-flow through cyclones that separate the two phases each time.

The tower modelled here is MOK's 4-stage preheater with an in-line **precalcliner** (PC). From top to bottom the train is:

- **Top cyclone** — raw-meal inlet, ambient temperature.
- **CY3 (stage 3)** — gas ≈ 600 °C, meal leaves this cyclone ~ 520 °C.
- **CY4 (stage 4)** — gas ≈ 760 °C, meal leaves ~ 700 °C.
- **PC (precalcliner)** — the big one. A second burner fires here (tertiary air from the cooler) so the meal temperature rises to ~ 870 °C and the endothermic reaction $\text{CaCO}_3 \rightarrow \text{CaO} + \text{CO}_2$ (**calcination**) runs to ~ 90 % completion *before* the meal enters the kiln. Calcination absorbs ~ 1.8 MJ/kg CaCO_3 ; doing it upstream of the kiln flame lets the kiln be smaller, hotter and more fuel-efficient.
- **Rotary kiln** (not modelled) — fuses the calcined meal to clinker at ~ 1450 °C.

This simulator tracks **three lumped stages** — PC, CY4, CY3 — because these are the sensors the plant publishes (`g_ph_cy4_gol_temp1`, `g_ph_cy3_gol_temp`, `g_ph_cy2_gol_temp`) and the lab-measured degree of calcination (`s_ph_sol_doc`, sparse ~ 8 h grab samples) is taken at the PC exit. The two top cyclones are folded into the CY3 boundary condition.

Physical flow picture:

- **Gas flows upward:** kiln $\rightarrow$ PC $\rightarrow$ CY4 $\rightarrow$ CY3 $\rightarrow$ stack.

- **Raw meal falls downward:** silo → CY3 → CY4 → PC → kiln.
- **Fuel enters at PC** (coal + RDF + tyres) and at the **kiln** (coal + RDF + oil + waste oil + nat gas + sludge + liquid AF).
- **Tertiary air** from the clinker cooler enters the PC to supply oxidiser for the PC burner.
- The **ID fan** at the top of CY3 pulls gas through the entire train.

1. What the branch did

The `extend-bagterp` branch extends what was previously a pure thermal model (three coupled temperature ODEs with fuel / air / ambient / gas-exchange / material-sink terms) with full **chemistry and mass coupling**. The goal was to close the loop from fuel inputs → temperatures → calcination rate → CO₂ release → mass-loss feedback on downstream heat sinks.

The net result: the simulator gained three new ODE states (DoC at each stage) and three algebraic outputs (CO₂ mole fractions), a sixth heat-flow module (`CalcinationKineticsModule`), a joint temperature + DoC fitting harness, and an interactive web report.

2. Internal primitives — the thermodynamic model

The preheater is modelled as **three lumped volumes** in series — PC at the bottom, CY4 above it, CY3 above that. Each volume is treated as well-mixed: one temperature, one degree-of-calcination (DoC), one CO₂ mole fraction. Heat flows and mass flows between stages are algebraic; temperatures and DoCs are integrated in time by explicit Euler at `dt_mins = 5` (simulation default) or `30` (fitting / reporting default).

2.1 Notation

Subscripts $s \in \{\text{pc}, \text{cy4}, \text{cy3}\}$ index the three stages, ordered along the material flow direction. Temperatures are in °C, DoC in $[0, 1]$, CO₂ in mole fraction.

Independent variables & inputs

Symbol	Meaning	Units	Tag / source
t	time	min (step Δt)	—
$Q_{\text{pc}}(t)$	PC fuel power (sum of coal + RDF + tyres × NCV)	GJ/h	derived from <code>f_pc_*</code> tputs
$Q_{\text{k}}(t)$	kiln fuel power	GJ/h	derived from <code>f_k_*</code> tputs
$T_{\text{kiln}}(t)$	kiln exit gas temperature (boundary condition)	°C	<code>g_k_pyro_temp</code>
$u_{\text{fan}}(t)$	ID-fan speed	%	<code>g_ph_gol_fan_speed</code>
$\dot{m}_{\text{in}}(t)$	raw-meal throughput entering CY3	t/h	<code>s_ph_sil_tput</code>
T_{amb}	ambient temperature	°C (20)	constant
$T_{\text{mat,in}}$	raw-meal inlet temperature at CY3	°C (80)	constant
R	gas constant	8.314 J/(mol·K)	—

State variables (integrated in time)

Symbol	Meaning	Units	Initial condition
T_s	gas temperature at stage s	°C	from first real sample
ξ_s	degree of calcination at stage s (DoC)	—	config: $\xi_{\text{pc}} = 0.85$, $\xi_{\text{cy4}} = 0.40$, $\xi_{\text{cy3}} = 0.10$

Algebraic (recomputed each step)

Symbol	Meaning	Units
$y_s^{\text{CO}_2}$	CO ₂ mole fraction in the gas leaving stage s upward	—
P_s^{eq}	CO ₂ equilibrium partial pressure at stage s (Baker 1962)	atm
σ_s	equilibrium suppression factor $\in [0, 1]$	—
k_s	Arrhenius rate constant at T_s	1/h

Symbol	Meaning	Units
ω_s	material transit rate $\dot{m}_{in} \phi_s / I_s$	1/h
ϕ_s	mass fraction of the raw-meal stream reaching stage s after upstream CO ₂ loss	—
T_s^{mat}	estimated material temperature entering stage s	°C

Fixed physical constants

Symbol	Meaning	Default
f_{CO2}	CO ₂ mass fraction of CaCO ₃ (44/100)	0.44
ΔH_{calc}	calcination enthalpy	1.794 MJ/kg CaCO ₃
E_a	activation energy for CaCO ₃ decomposition	175.7 kJ/mol (default); 149.2 kJ/mol (DoC-fit)
$c_{p,air}$	air specific heat	1.05 kJ/(kg·K)
$c_{p,mat}$	meal specific heat	0.9 kJ/(kg·K)
ρ_a	hot-gas density	0.4 kg/m ³

Fitted scalar parameters

The simulator exposes **seven** parameters to the thermal optimiser (`fit_normal_params.py`) and **nine** to the kinetics optimiser (`fit_calcination_params.py`); the two are fit on different objectives (see §3).

Symbol	Role	Param name
α_{pc}, α_k	effective fuel-output slopes (GJ/h of heat into PC per GJ/h of fuel input)	<code>pc_fuel_slope</code> , <code>kiln_fuel_slope</code>
C_{th}	effective PC thermal mass	<code>thermal_mass_effective</code>
$k_{pc \rightarrow cy4}, k_{cy4 \rightarrow cy3}$	inter-stage gas-flow heat-transfer coefficients	<code>k_pc_to_cy4</code> , <code>k_cy4_to_cy3</code>
β	overall material heat-sink scale	<code>material_cooling_scale</code>
$k_{amb,*}, \tau_*$	time-varying ambient loss coefficients (warm-up vs cool-down, early vs late)	<code>k_ambient_{early,late,warmup_early,warmup_late}</code> , <code>tau_k_{transition,warmup}</code>
A, E_a	Arrhenius lumped pre-exponential & activation energy	<code>effective_pre_exponential</code> , <code>activation_energy</code>
f_{ls}	limestone mass fraction in raw meal	<code>limestone_fraction</code>
I_{pc}, I_{cy4}, I_{cy3}	solid inventory held at each stage	<code>stage_inventory_*</code>
ξ_0	initial DoC of meal entering CY3	<code>initial_doc</code>
y_{kiln}^{CO2}	CO ₂ mole fraction in kiln-exit gas arriving at PC	<code>co2_kiln_gas</code>
$\dot{V}_g^{eff}$	effective gas-flow capacity (mass equivalent)	<code>gas_flow_effective</code>

2.2 State vector

With kinetics enabled the full state per step is a 6-vector (3 temperatures + 3 DoCs). CO₂ mole fractions are algebraic (closed-form, §2.5):

$$\mathbf{Y}(t) = [T_{pc}, T_{cy4}, T_{cy3}, \xi_{pc}, \xi_{cy4}, \xi_{cy3}]$$

The exogenous driver per timestep is:

$$\mathbf{u}(t) = [Q_{pc}(t), Q_k(t), T_{kiln}(t), u_{fan}(t), \dot{m}_{in}(t)]$$

2.3 Heat-flow modules

The thermal RHS is built by summing contributions from six independent modules, each implementing the `HeatFlowModule` Protocol. Modules can be independently ablated through `SimulationConfig.enable_*` flags.

(a) Fuel heating — `FuelHeatingModule`

All fuel heat is deposited in PC. CY4 and CY3 only see it after it propagates upward through the gas-flow module. The effective power is linear in fuel input with a saturation cap:

$$\dot{Q}_{\text{fuel}} = \min(\alpha_{\text{pc}} Q_{\text{pc}}, Q_{\text{pc,max}}) + \min(\alpha_{\text{k}} Q_{\text{k}}, Q_{\text{k,max}})$$

$$\left. \frac{dT_{\text{pc}}}{dt} \right|_{\text{fuel}} = \frac{\dot{Q}_{\text{fuel}}}{C_{\text{th}}}$$

(b) Ambient loss — `AmbientLossModule`

Linearised shell loss with a *phase-dependent* coefficient. During cool-down the external skin radiates freely; during warm-up the internal refractory is absorbing rather than leaking. The effective $k(t)$:

$$k(t) = \begin{cases} k_{\text{warmup,late}} + (k_{\text{warmup,early}} - k_{\text{warmup,late}}) e^{-\Delta t_{\text{since restart}} / \tau_{\text{warmup}}} & \text{warm-up} \\ k_{\text{late}} + (k_{\text{early}} - k_{\text{late}}) e^{-t / \tau_{\text{trans}}} & \text{cool-down} \end{cases}$$

$$\left. \frac{dT_s}{dt} \right|_{\text{amb}} = -k(t) (T_s - T_{\text{amb}})$$

(c) Gas-flow exchange — `GasFlowExchangeModule`

Gas flows *upward*: kiln $\rightarrow$ PC $\rightarrow$ CY4 $\rightarrow$ CY3. Each stage heats the one above it and cools itself by the same rate (energy conservation). The kiln-end term is gated to zero because the measured `g_k_pyro_temp` is too noisy:

$$\left. \frac{dT_{\text{pc}}}{dt} \right|_{\text{gas}} = k_{k \rightarrow \text{pc}} [T_{\text{kiln}} - T_{\text{pc}}]_+ - k_{\text{pc} \rightarrow \text{cy4}} [T_{\text{pc}} - T_{\text{cy4}}]_+$$

$$\left. \frac{dT_{\text{cy4}}}{dt} \right|_{\text{gas}} = k_{\text{pc} \rightarrow \text{cy4}} [T_{\text{pc}} - T_{\text{cy4}}]_+ - k_{\text{cy4} \rightarrow \text{cy3}} [T_{\text{cy4}} - T_{\text{cy3}}]_+$$

$$\left. \frac{dT_{\text{cy3}}}{dt} \right|_{\text{gas}} = k_{\text{cy4} \rightarrow \text{cy3}} [T_{\text{cy4}} - T_{\text{cy3}}]_+$$

The $[\cdot]_+$ clamp prevents gas flowing backward through the tower when transient perturbations flip a temperature gradient.

(d) Air cooling — `AirCoolingModule`

Convective cooling from the ID fan pulling ambient air. Air mass flow scales linearly with fan speed; the heat sink is distributed across the three stages with fixed mass fractions 50/30/20 %:

$$\dot{m}_{\text{air}} = u_{\text{fan}} k_{\text{fan}} \rho_a \text{ [kg/h]}$$

$$\dot{Q}_{\text{air}} = \dot{m}_{\text{air}} c_{p,\text{air}} (\bar{T} - T_{\text{amb}}) \gamma_{\text{air}}, \quad \bar{T} = \frac{1}{3} \sum_s T_s$$

$$\left. \frac{dT_s}{dt} \right|_{\text{air}} = -\frac{\dot{Q}_{\text{air}} \phi_s^{\text{air}}}{C_{\text{th}}^{\text{air}}}, \quad \phi^{\text{air}} = [0.5, 0.3, 0.2]$$

how about no ^

(e) Material heat sink — `MaterialHeatSinkModule`

Raw meal descends top-down. At each stage the meal enters at T_s^{mat} (the exit temperature of the stage above) and is heated toward T_s with efficiency η_{HX} . Mass flow is reduced by calcination: $\phi_s = 1 - f_{\text{ls}} \xi_s f_{\text{CO2}}$:

$$\dot{Q}_s^{\text{mat}} = \dot{m}_{\text{in}} \phi_s c_{p,\text{mat}} (T_s - T_s^{\text{mat}})$$

$$T_{\text{cy3}}^{\text{mat}} = T_{\text{mat,in}}, \quad T_{\text{cy4}}^{\text{mat}} = T_{\text{cy3}}^{\text{mat}} + \eta_{\text{HX}} (T_{\text{cy3}} - T_{\text{cy3}}^{\text{mat}}), \quad T_{\text{pc}}^{\text{mat}} = T_{\text{cy4}}^{\text{mat}} + \eta_{\text{HX}} (T_{\text{cy4}} - T_{\text{cy4}}^{\text{mat}})$$

Below a `material_flow_threshold` = 100 t/h the module returns zero — the sink turns off cleanly at plant idle, which is what makes shutdown events well-posed (the temperature ODE collapses to pure exponential cool-down).

(f) Calcination kinetics — `CalcinationKineticsModule`

Described in §2.5. Computes $d\xi_s/dt$ and the endothermic heat sink $\dot{Q}_s^{\text{calc}}$ using an analytical integration step so the integrator stays stable even when $k(T) \gg 1/\Delta t$.

2.4 Temperature ODE

Summing all six module contributions gives one coupled ODE per stage:

$$\frac{dT_s}{dt} = \frac{\dot{Q}_{\text{fuel}} 1_{s=\text{pc}}}{C_{\text{th}}} - k(t) (T_s - T_{\text{amb}}) + \text{GasFlow}_s(\mathbf{T}) - \frac{\dot{Q}_{\text{air}} \phi_s^{\text{air}}}{C_{\text{th}}^{\text{air}}} - \frac{\beta \dot{Q}_s^{\text{mat}}}{C_{\text{th}}^{\text{mat}}} - \frac{\dot{Q}_s^{\text{calc}}}{C_{\text{th}}^{\text{calc}}}$$

Each module carries its own `thermal_mass_effective` divisor so the same GJ/h can be mapped to different °C/h depending on what's driving the change. This is the dominant source of over-parameterisation in the model.

2.5 Calcination kinetics detail

Three things happen per stage per timestep.

(i) Arrhenius rate

$$k_s = A \exp\left(-\frac{E_a}{R(T_s + 273.15)}\right) \quad [1/\text{s}]$$

(ii) CO₂-equilibrium suppression (Baker 1962)

Calcination is reversible: at high CO₂ partial pressure, CaO back-reacts with CO₂ and calcination stops. The equilibrium pressure:

$$\log_{10} P_s^{\text{eq}} = 7.079 - \frac{8308}{T_s + 273.15} \quad [\text{atm}]$$

Gas flows bottom-up. The CO₂ at PC is the kiln boundary $y_{\text{kiln}}^{\text{CO}_2}$; at CY4 it's the CO₂ leaving PC, etc. The reaction both generates CO₂ and is suppressed by it — a coupled system solved in closed form:

$$\sigma_s = \frac{1 - y_{s-1}^{\text{CO}_2}/P_s^{\text{eq}}}{1 + k_s(1 - \xi_s)h_s/P_s^{\text{eq}}}, \quad y_s^{\text{CO}_2} = y_{s-1}^{\text{CO}_2} + h_s k_s(1 - \xi_s) \sigma_s$$

$$h_s = \frac{I_s f_{\text{ls}}}{\dot{V}_g^{\text{eff}}}$$

Bigger stages with more CaCO₃ generate more CO₂ per unit reaction, suppressing the reaction more. The fitted I_s values encode both residence time *and* chemical back-pressure.

(iii) Analytical DoC integration

With the suppressed rate $k_s^{\text{eff}} = k_s \sigma_s$ and material transit rate $\omega_s = \dot{m}_{\text{in}} \phi_{s-1}/I_s$, each stage's DoC ODE is linear:

$$\frac{d\xi_s}{dt} = k_s^{\text{eff}} (1 - \xi_s) + \omega_s (\xi_{s-1} - \xi_s)$$

Closed-form step:

$$\xi_s^{\text{ss}} = \frac{k_s^{\text{eff}} + \omega_s \xi_{s-1}}{k_s^{\text{eff}} + \omega_s}, \quad \xi_s(t + \Delta t) = \xi_s^{\text{ss}} + (\xi_s(t) - \xi_s^{\text{ss}}) e^{-(k_s^{\text{eff}} + \omega_s) \Delta t}$$

Unconditionally stable no matter how large k_s^{eff} gets — critical, because at 870 °C the Arrhenius rate easily exceeds 10² 1/h and explicit-Euler would require $\Delta t \lesssim 1$ s. The integrated CaCO₃ consumed over the timestep (which drives the

endothermic heat sink) is computed analytically from the same closed form:

$$\Delta m_s^{\text{calc}} = I_s f_{\text{ls}} k_s^{\text{eff}} \left[(1 - \xi_s^{\text{ss}}) \Delta t + \frac{\xi_s(t) - \xi_s^{\text{ss}}}{k_s^{\text{eff}} + \omega_s} \left(e^{-(k_s^{\text{eff}} + \omega_s) \Delta t} - 1 \right) \right]$$

$$\dot{Q}_s^{\text{calc}} = \frac{\Delta m_s^{\text{calc}} \Delta H_{\text{calc}}}{\Delta t}$$

2.6 One consolidated ODE

With $N_T = 3$ temperatures and $N_\xi = 3$ DoCs, the full system is a 6-D ODE with algebraic CO₂ closure:

$$\dot{\mathbf{Y}} = \begin{bmatrix} \dot{T}_s \\ \dot{\xi}_s \end{bmatrix} = \begin{bmatrix} \text{sum of six module contributions (§2.4)} \\ k_s^{\text{eff}}(1 - \xi_s) + \omega_s(\xi_{s-1} - \xi_s) \end{bmatrix}_{s \in \{\text{pc}, \text{cy4}, \text{cy3}\}}$$

The temperature block uses explicit Euler with a 5-minute (or 30-minute) step; the DoC block uses the analytical closed-form step from §2.5 (iii). The CO₂ system is solved once per timestep bottom-up, before both integrations.

2.7 Code layout — classes and methods

All simulator code lives in a single file: `exp/users/michael/physics_simulator/heat_transfer_model.py`.

Role	Class / function	Line	Responsibility
Main driver	<code>HeatTransferSimulator</code>	790	Holds a <code>SimulationConfig</code> , instantiates modules in <code>_init_modules</code> , exposes <code>simulate(...)</code> .
Config root	<code>SimulationConfig</code>	753	Pydantic tree: dt, ambient T, fuel threshold, per-module params, enable flags, initial DoC.
State type	<code>StageTemperatures</code>	15	Three floats: <code>t_pc</code> , <code>t_cy4</code> , <code>t_cy3</code> .
State type	<code>StageCalcination</code>	23	Three DoCs + three CO ₂ mole fractions per stage.
State type	<code>SimulationState</code>	71	Bundles temps + DoC + time + warm-up phase flag.
Contribution	<code>HeatFlowContribution</code>	39	Per-stage dT/dt plus total Q; overloads <code>+</code> / <code>-</code> .
Protocol	<code>HeatFlowModule</code>	92	<code>compute(state, inputs) → HeatFlowContribution</code> .
Module	<code>AmbientLossModule</code>	228	§2.3 (b) time-varying shell loss.
Module	<code>FuelHeatingModule</code>	268	§2.3 (a) linear+capped fuel-to-PC term.
Module	<code>AirCoolingModule</code>	305	§2.3 (d) fan-driven convective loss.
Module	<code>MaterialHeatSinkModule</code>	359	§2.3 (e) meal sensible heat, DoC-aware mass fractions.
Module	<code>GasFlowExchangeModule</code>	468	§2.3 (c) upward gas transport between stages.
Module	<code>CalcinationKineticsModule</code>	511	§2.5 Arrhenius + CO ₂ equilibrium + analytical DoC step.
Kinetics internal	<code>_arrhenius_rate</code>	607	$Ae^{-E_a/RT}$ in 1/s.
Kinetics internal	<code>_co2_equilibrium_pressure</code>	540	Baker 1962 correlation.
Kinetics internal	<code>_stage_co2_and_suppression</code>	565	Closed-form CO ₂ /suppression per stage (§2.5 ii).
DoC integration	<code>_integrate_stage</code>	617	Analytical linear-ODE step (§2.5 iii).
DoC integration	<code>CalcinationKineticsModule.compute</code>	657	Ties Arrhenius + CO ₂ + transit + heat sink together.

`HeatTransferSimulator.simulate(...)` per-step flow

1. Clamp and fill inputs (NaN kiln temp → previous PC, negative fan/flow → 0).
2. Detect phase: if total fuel just crossed the `fuel_threshold` from below, mark `in_warmup_phase = True` and record `fuel_restart_time`.
3. Build `SimulationState` (previous temps, previous DoC, time, phase) and `SimulationInputs`.

4. Sum module contributions. Each module returns a `HeatFlowContribution`; the sum is the total dT/dt for each stage.
5. Advance DoC by reading `calcination_module.next_calcination`.
6. Euler-integrate temperatures: $T_s^+ = T_s + \Delta t \cdot \dot{T}_s$, then clip to $[T_{\text{amb}}, 1200 \text{ }^\circ\text{C}]$.
7. Record per-module Q_{total} into `q_<name>` output arrays for diagnostics.

3. How the model is evaluated

3.1 Startup / shutdown ablations — `evaluate.py`

`find_events_with_material_transition` scans 2024-05 through 2024-11 MOK data for contiguous runs of `s_ph_sil_tput < 100` t/h lasting ≥ 6 h, and extends each event by up to 4 hours *after* feed resumes so the startup transient is captured. `run_single_module_ablations` then runs the full model and six single-module-ablated models on each event, comparing average of $|\hat{T}_{\text{pc}} - T_{\text{pc}}|$ across the three stages against the measured `g_ph_cy*gol_temp*` traces.

3.2 Normal-operation thermal fit — `fit_normal_params.py`

`find_normal_operation_windows` returns sustained (≥ 24 h) windows with $\dot{m}_{\text{in}} \geq 100$ t/h and at least 3 non-NaN DoC lab readings. The fitter runs the simulator in **online mode**: each window is split into `CHUNK_HOURS = 12` chunks and at each chunk boundary the three temperatures are re-initialised from the real plant data, then integrated forward for 12 h. The first `BURNIN_STEPS = 4` (2 h) of each chunk are skipped when computing MAE.

SciPy `differential_evolution` (popsize default, `maxiter=200`, `seed=42`) optimises a joint objective over 7 parameters:

```
obj = mean(T_MAE over windows) + 0.5 * mean(DoC_MAE over windows)
```

with parameters `pc_fuel_slope`, `kiln_fuel_slope`, `thermal_mass_effective`, `k_pc_to_cy4`, `k_cy4_to_cy3`, `material_cooling_scale`, `log10(effective_pre_exponential)`. Fuel cap is disabled during fitting (`FUEL_MAX_UNCAPPED = 1e6`).

Baseline context from the file header: "During normal running the temperature error is ~ 290 $^\circ\text{C}$, which means DoC can't work." That ~ 290 $^\circ\text{C}$ is the aggregated T-MAE of the unfitted defaults over the 10 selected normal windows — it motivates the whole normal-operation fit.

3.3 DoC evaluation — `evaluate_doc.py`

Same windows, same chunked online mode, same burn-in — but the metric is DoC (%). The model's `doc_exit_pct` at each non-NaN `s_ph_sol_doc` timestamp is compared against the lab value. Reports Sim MAE, Persistence MAE (naive "today's reading = tomorrow's"), and Pearson correlation.

3.4 Calcination-only rollout — `evaluate_doc_driven.py` / `plot_fitted_rollout.py`

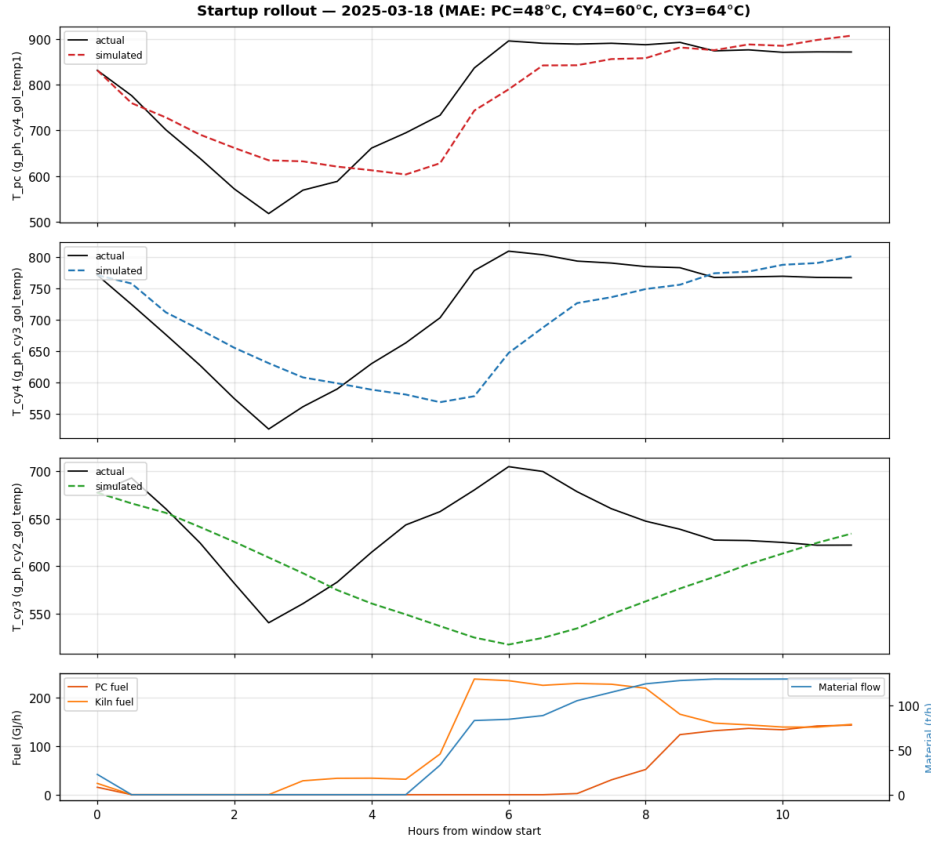
Decouples chemistry from the thermal model by feeding *measured* T_s directly into the calcination kinetics. Useful for fitting A , E_a , f_{is} , I_s , ξ_0 , $y_{\text{kiln}}^{\text{CO}_2}$, $\dot{V}_g^{\text{eff}}$ in isolation.

4. Temperatures — startup and steady state

Typical amplitudes, read from the plot-label averages in `plot_fitted_rollout.py` and the scenario selector in `generate_report.py`:

Stage	Steady-state avg	Typical span in normal window	Floor during full shutdown
<code>t_pc</code>	871 $^\circ\text{C}$	830–900	< 200
<code>t_cy4</code>	763 $^\circ\text{C}$	720–800	< 200
<code>t_cy3</code>	602 $^\circ\text{C}$	570–640	< 200

4.1 Startup — the three-phase transient



2025-03-18 startup, 11 h window. Black: measured `g_ph_cy4_gol_temp1` (top), `g_ph_cy3_gol_temp` (middle), `g_ph_cy2_gol_temp` (bottom). Dashed: simulator prediction from a single initial condition (no chunked re-init) driven by real fuel + kiln-temp + material-flow inputs. Bottom panel shows PC fuel, kiln fuel, and raw-meal throughput. Fit config:

`evaluate_doc.py::fitted_config()`. Generated by `plot_summary_rollouts.py`.

A startup in this tower is a fundamentally different creature from a cooler startup: there is no radiative boundary condition pre-warming the gas space before feed returns, and the refractory mass of the tower is much smaller per unit gas volume than a 30-m cooler. The transient is dominated by fuel energy going *directly* into gas sensible heat, with the three stages responding at three different timescales set by the gas-flow cascade:

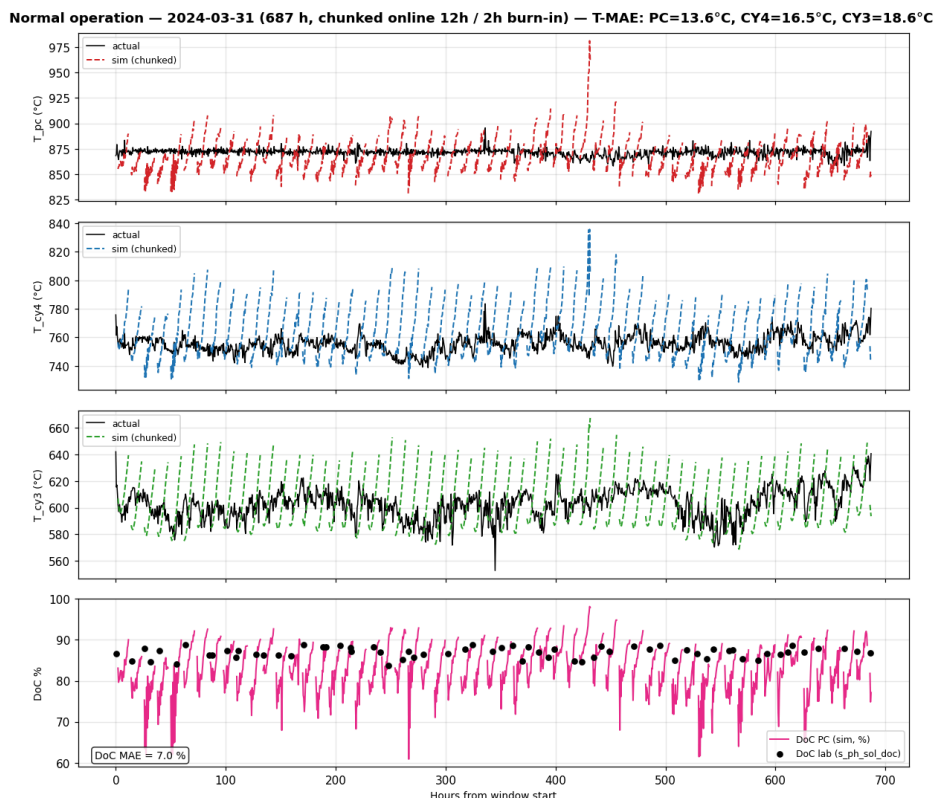
- Phase 1 — PC lights off.** PC fuel turns on before raw meal starts falling. T_{pc} climbs first because 100 % of fuel heat deposits at PC. With fuel threshold = 30 GJ/h, the warm-up branch of the ambient-loss k is triggered and the effective k decays from `k_ambient_warmup_early` = 0.072 down to 0.024 /h over τ_{warmup} = 4 h. PC time constant during this phase is $C_{th}/(\dot{Q}_{fuel}/\Delta T) \approx 1\text{--}2$ h.
- Phase 2 — gas cascade fills CY4 then CY3.** The `GasFlowExchangeModule` passes $k_{pc \rightarrow cy4} \sim 1\text{--}3$ °C/h/°C of gradient to CY4, which in turn passes $k_{cy4 \rightarrow cy3} \sim 0.4\text{--}1.5$ °C/h/°C to CY3. Characteristic rise times are roughly the inverse — CY4 ramps over ~ 1 h, CY3 over $\sim 2\text{--}3$ h. DoC is still essentially frozen at its initial condition because the Arrhenius rate doesn't light off below ~ 650 °C.
- Phase 3 — raw-meal feed returns.** When $\dot{m}_{in}$ crosses 100 t/h, the `MaterialHeatSinkModule` switches on and starts pulling sensible heat into the falling meal. Simultaneously k_s^{eff} in the PC starts ramping because T_{pc} is already ≥ 800 °C; calcination ignites and the endothermic heat sink $\dot{Q}_s^{calc}$ adds to the load. Both effects produce a visible “knee” in T_{pc} — the curve flattens or slightly dips before fuel overrides it again. Lab DoC lags this ignition by one ω_s^{-1} residence time ($\sim 5\text{--}10$ min for the fitted I_{pc}).

The model captures the shape but lags the real PC temperature recovery by roughly 1 h once fuel comes back on at $t \approx 5$ h, and it under-predicts the CY3 temperature rise during Phase 3 — consistent with the `k_cy4_to_cy3` coupling being too weak

in the current fit.

4.2 Steady state

Steady state in this tower is *not* steady temperatures; it's small-amplitude quasi-periodic oscillation around the ~871/763/602 set-point driven by three disturbance sources: fuel-rate micro-adjustments from the kiln operator ($\sim \pm 5$ GJ/h), material-flow swings as the silo feeder cycles ($\sim \pm 10$ t/h), and CY3 inlet CO₂ drift when the kiln oxygen tracking is active.



2024-03-31 — 687 h of normal operation, the longest contiguous window in the fitting set (66 DoC lab samples). Top three rows: actual (black) vs. simulated (dashed colour) temperatures, re-initialised from real sensor values every 12 h with the first 2 h of each chunk elided (burn-in). Bottom row: simulator DoC at PC exit (pink line) vs lab `s_ph_sol_doc` grab samples (black dots). Generated by `plot_summary_rollouts.py`.

Current performance against `evaluate_doc.py::fitted_config()` on this data snapshot (2024-01 to 2026-01):

Scenario	Window	T-MAE PC (°C)	T-MAE CY4 (°C)	T-MAE CY3 (°C)	DoC MAE (%)
Startup (single initial condition)	2025-03-18 (11 h)	48	60	64	—
Normal (chunked online, 12 h / 2 h burn-in)	2024-03-31 (687 h, 66 DoC)	13.6	16.5	18.6	7.0
Baseline (unfitted defaults, normal)	10-window aggregate	~ 290 combined-stage (per <code>fit_normal_params.py</code> header)	—	—	—

On the DoC-only harness (`plot_fitted_rollout.py`, measured temps feeding the fitted kinetics), 8 windows:

Window	Sim MAE (%)	Persist MAE (%)	Skill (1 – sim/persist)
2024-01-01	1.2	1.3	+0.06

Window	Sim MAE (%)	Persist MAE (%)	Skill (1 – sim/persist)
2024-03-31	1.2	1.4	+0.16
2024-04-29	1.0	0.9	−0.01
2024-07-12	1.2	1.6	+0.23
2024-08-09	1.4	1.5	+0.10
2025-06-20	1.1	0.5	−1.33
2025-08-17	1.1	1.2	+0.06
2025-10-19	0.9	1.0	+0.10

The DoC MAE numbers here (~ 1 % absolute) are an order of magnitude lower than the thermally-coupled 7.0 % from the normal-operation rollout above because the two harnesses answer different questions: the DoC-only harness uses *measured* temperatures so chemistry error is isolated from thermal error. The 2025-06-20 window is instructive: persistence MAE drops to 0.5 % (DoC is almost flat), and the simulator can't beat a near-perfect naive baseline. In windows with real DoC movement (2024-07-12, 2024-03-31), skill is convincingly positive.

Why steady-state T-MAE is an order of magnitude better than startup T-MAE, even though both are fitted on the same loss: during normal running the six modules are operating in their linear regime (small perturbations around a stable manifold), whereas a startup crosses hard non-linearities (fuel threshold, DoC activation energy, material-flow threshold, warm-up vs cool-down k). The fitter trades off some startup accuracy for a clean steady state.

5. Fitted parameter values

The branch ends with two independent fit results, each from a different optimisation.

5.1 Thermal parameters — from `evaluate_doc.py::fitted_config()`

Output of `fit_normal_params.py` (DE, 7 params, `seed=42`, `maxiter=200`) on 10 normal-operation windows with the joint T + 0.5×DoC objective:

Parameter	Default	Fitted	Bounds	Role
<code>pc_fuel_slope</code>	5.93	20.96	[0.5, 30]	effective GJ/h heat into PC per GJ/h fuel input — default undershot normal-running fuel effectiveness.
<code>kiln_fuel_slope</code>	6.15	19.47	[0.5, 30]	same story for the kiln term.
<code>thermal_mass_effective</code>	2.79	13.06	[0.3, 15]	near upper bound — optimiser wants more thermal inertia to soak up the larger fuel slopes.
<code>k_pc_to_cy4</code>	3.3	1.20	[0.1, 15]	gas-exchange coefficient; lower than default.
<code>k_cy4_to_cy3</code>	1.5	0.36	[0.1, 10]	same.
<code>material_cooling_scale</code>	1.00	0.06	[0.05, 5]	near lower bound — material sink heavily attenuated in normal running.
<code>effective_pre_exponential</code>	1e7	1.23×10^7	$[10^5, 10^{13}]$	close to Arrhenius prior.

5.2 Calcination-only parameters — from `generate_report.py::FITTED`

Output of `fit_calcination_params.py` driven by measured temperatures:

Parameter	Default	Fitted	Role
<code>effective_pre_exponential</code>	1×10^7	1.04×10^5	two orders below the thermal fit — pointing to modelling error in the thermal half.
<code>activation_energy</code>	175.7 kJ/mol	149.2 kJ/mol	below literature for pure CaCO_3 ; soaks up residence-time and mass-transfer effects.
<code>limestone_fraction</code>	0.42	0.260	much lower than the nominal MOK raw-meal CaCO_3 fraction; compensating for a too-high A .
<code>stage_inventory_pc</code>	3.0 t	12.32 t	larger PC hold-up — longer residence lets CO_2 equilibrium catch up with Arrhenius ignition.

Parameter	Default	Fitted	Role
<code>stage_inventory_cy4</code>	2.0 t	0.50 t	CY4 essentially a flow-through.
<code>stage_inventory_cy3</code>	2.0 t	4.48 t	CY3 holds more than default.
<code>initial_doc</code>	0.05	0.192	meal enters CY3 already 19 % calcined — consistent with two unmodelled top cyclones doing real work.
<code>co2_kiln_gas</code>	0.25	0.052	implausibly low; likely compensating for unmodelled tertiary-air dilution.
<code>gas_flow_effective</code>	1500 t/h eq.	4872.7 t/h eq.	high gas capacity → small h_s → weak self-suppression; compensating for too-eager Arrhenius.

6. Open issues and candidate next steps

- **Two disagreeing fits for A .** The thermal fit wants $A \approx 1.2 \times 10^7$ and the DoC-driven fit wants $A \approx 10^5$. That's a two-order gap — a strong signal that the thermal model and the chemistry model are compensating for each other's errors. A joint T + DoC fit over *all* 16 parameters at once would expose the inconsistency in the loss surface.
- **Several kinetics parameters look unphysical.** $f_{is} = 0.26$ is well below the real raw-meal CaCO_3 content of MOK feed (~ 0.75 in lab data); $y_{\text{kiln}}^{\text{CO}_2} = 0.05$ is implausibly low for kiln exit gas. Fixing f_{is} from mineralogy and $y_{\text{kiln}}^{\text{CO}_2}$ from gas-analysis data would collapse two dimensions of slack.
- **Three-term ambient loss is probably over-parameterised.** 6 degrees of freedom for what is fundamentally one effect. Reducing to a single $k(t)$ with one time constant (and asymmetric sign depending on fuel phase) would flatten the loss surface.
- **Material flow threshold is a hard discontinuity.** At $\dot{m}_{\text{in}} = 99$ t/h the material sink is off; at 101 t/h it's fully on. Puts a kink in the RHS right where startup events live. A smooth sigmoid with ~ 10 t/h width would be more physical and differentiable.
- **DoC persistence is hard to beat.** Lab DoC varies slowly (~ 8 h sampling, autocorrelated over days); the simulator predicts step-to-step variation washed out by lab-measurement noise. The model only earns skill during actual DoC excursions.
- **Kiln-gas coupling is gated off.** `k_kiln_to_pc = 0` because `g_k_pyro_temp` is too noisy. Investigating whether a denoised / LP-filtered kiln temp unlocks real skill at the PC end is the single biggest latent improvement; kiln-exit gas carries ~ 40 % of the tower's total energy input.
- **Add advection** to more correctly represent heat transfer.
- **Single-file refactor.** `heat_transfer_model.py` is now 1007 lines. Splitting modules into a `modules/` subpackage would make the Protocol's intent much clearer. `fit_thermal_params.py` still imports from a now-deleted `heat_flow_simulator` module and should be removed or ported.

Generated from branch `extend-bagterp`, 7 commits on top of `main`. Source tree: `exp/users/michael/physics_simulator/`.
Companion: interactive `report_heat_flow_live.html`, static rollouts `startup_rollout.png` / `normal_rollout.png`.

Ideally we would be able to scrutinise the equations somewhere where we could comment on them, so I will try and generate that structured output next

Cooler progress summary

Again generated with claude

Branch: `worktree-cooler-first-principles` · 11 commits on top of `main` · Source tree: `exp/users/michael/physics_simulator/cooler`

[cooler_summary.html](#)

▼ MOK Grate Cooler — First-Principles Simulator

0. What is a grate cooler?

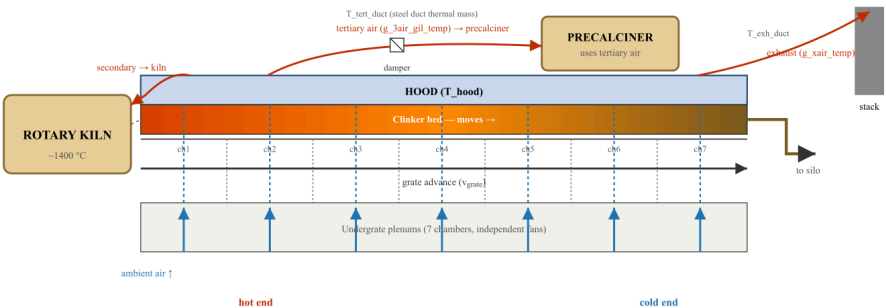
A **grate cooler** is the heat exchanger that sits between the rotary kiln and the clinker silo in a cement plant. Its job is to quench the white-hot clinker ($\approx 1400\text{ }^{\circ}\text{C}$) leaving the kiln down to around $100\text{ }^{\circ}\text{C}$ so it can be handled and ground into cement — and, crucially, to recover the sensible heat in the air so it can be re-used by the combustion processes upstream.

Clinker falls from the kiln discharge onto a horizontal grate made of perforated plates. The grate advances in steps, sliding the clinker the length of the cooler ($\sim 30\text{ m}$) over roughly 20–30 minutes. Underneath the grate, a bank of independently-controlled fans forces ambient air *upward* through the clinker bed. The hot air above the grate is then split into three streams:

- **Secondary air** — drawn back into the kiln through the hood, to sustain kiln-flame combustion ($\approx 800\text{--}1100\text{ }^{\circ}\text{C}$).
- **Tertiary air** — ducted up from the hood to the *precalciner*, a second burner that calcines the raw meal ($\approx 600\text{--}900\text{ }^{\circ}\text{C}$).
- **Exhaust air** — the remaining cooler air, vented through bag filters to stack ($\approx 100\text{--}300\text{ }^{\circ}\text{C}$).

At MOK there are **7 undergrate chambers** and **two grate sections** (the hot end and cold end run at independent speeds). The two primary sensors the model predicts are `g_3air_gil_temp` (tertiary air to precalciner) and `g_xair_temp` (exhaust air to stack).

[IMAGE: schematic.png — the SVG cooler schematic, export separately if desired]



1. What the branch did

The branch is a research iteration on a lumped-parameter physical model of the MOK grate cooler. Prior versions were steady-state-only and missed the characteristic three-phase transient seen in the tertiary-air signal `g_3air_gil_temp` at startup. The work on this branch reproduces that transient.

The programme was structured as six linked user stories (`prd.json`), all completed:

ID	Title	Outcome
COOL-01	Startup evaluation harness	Auto-detects zero-throughput → steady windows; computes startup-specific MAE; Oct 10 2024 chosen as the canonical test case.
COOL-02	Hood & duct thermal mass	Three new state variables <code>T_hood</code> , <code>T_tert_duct</code> , <code>T_exh_duct</code> with NTU-effectiveness air-to-wall exchange and shell losses.
COOL-03	Kiln-end radiation into hood	$\sigma \cdot VFA \cdot \epsilon \cdot (T_{kiln}^4 - T_{hood}^4)$ term driving hood temperature during shutdown; uses <code>g_k_pyro_temp</code> as radiation source.
COOL-04	Clinker inventory per chamber	Mass array <code>m_clinker[i,j]</code> integrated over time; thermal mass and bed height now derived from actual mass, not instantaneous throughput.
COOL-05	Dispersive transport	Throughput lag replaced with <i>shift + first-order low-pass</i> ; parameter <code>throughput_dispersion_mins</code> (default 10 min) smears the arrival wavefront.

ID	Title	Outcome
COOL-06	Parameter tuning	Differential-evolution fit over 17 parameters; combined startup-window MAE 186 → 69 vs pre-branch baseline.

Two further commits beyond the PRD: tertiary/exhaust airflow paths were separated in the hood model, and the convective solver was refactored from NTU-effectiveness to a direct energy balance ($Q = h_{\text{eff}} \cdot A_{\text{bed}} \cdot \Delta T$, capped by the air-side capacity rate). The refactor made `convective_scale` dimensionally meaningful (it is h_{eff} in $\text{W/m}^2\text{K}$) and simplified the sensitivity landscape for the optimiser.

2. Internal primitives — the thermodynamic model

The cooler is modelled as a 1-D plug-flow reactor with $N = 21$ cells (7 physical chambers $\times$ 3 sub-cells) plus three well-mixed wall tanks (hood, tertiary duct, exhaust duct). Each cell is treated as a *lumped volume*: inside the cell the clinker is at a single temperature, the air is at a single temperature, and heat-transfer rates are proportional to a bed surface area derived from geometry.

2.1 Notation

All symbols used in the equations below. Subscript $j \in \{1, \dots, N\}$ denotes a cell index; wall subscripts $w \in \{\text{hood}, \text{tert}, \text{exh}\}$. Temperatures are in $^{\circ}\text{C}$ unless raised to the fourth power (then Kelvin).

Independent variables & inputs

Symbol	Meaning	Units	Tag / source
t	time	min (step $\Delta t = 1$ min)	—
$u(t)$	exogenous driver vector	—	plant telemetry
$T_{\text{in}}(t)$	clinker inlet temp from kiln discharge	$^{\circ}\text{C}$	<code>g_k_gol_temp1</code>
$T_{\text{kiln}}(t)$	kiln radiation source temp (precalc pyro)	$^{\circ}\text{C}$	<code>g_k_pyro_temp</code>
$\dot{m}_{\text{in}}(t)$	raw-meal throughput (pre-lag)	t/h	<code>s_ph_sil_tput</code>
$\dot{V}_{a,j}(t)$	airflow through cell j	m^3/h	<code>g_c_chamberN_airflow</code> (apportioned)
$v_1(t), v_2(t)$	grate-1 / grate-2 speeds	%	<code>p_c_grate[2]_speed</code>
$d(t)$	tertiary-air damper position	%	<code>p_3air_damper_position</code>
T_{amb}	ambient temperature	$^{\circ}\text{C}$ (30)	constant
σ	Stefan-Boltzmann constant	$5.67\text{e-}8 \text{ W/m}^2\text{K}^4$	—

State variables (integrated in time)

Symbol	Meaning	Units	Initial condition
$\mathbf{Y}(t) \in \mathbb{R}^{2N+3}$	full state vector	—	see below
$T_{c,j}$	clinker temperature in cell j	$^{\circ}\text{C}$	interpolated from data
$m_{c,j}$	clinker mass in cell j	kg	$\dot{m}_{\text{in}}(0) \tau_j$
T_{hood}	hood-wall temperature	$^{\circ}\text{C}$	from first air sample
T_{tert}	tertiary-duct wall temperature	$^{\circ}\text{C}$	from first air sample
T_{exh}	exhaust-duct wall temperature	$^{\circ}\text{C}$	from first air sample
$\tilde{m}_{\text{in}}(t)$	lagged + dispersed throughput (auxiliary state)	t/h	$\dot{m}_{\text{in}}(0)$

Algebraic (recomputed each step from $\mathbf{Y}$ and u)

Symbol	Meaning	Units
$T_{a,j}$	air outlet temperature leaving cell j	$^{\circ}\text{C}$
$h_{\text{bed},j}$	clinker bed height in cell j	m

Symbol	Meaning	Units
$V_{\text{bed},j}$	bed volume in cell j	m^3
$A_{\text{bed},j}$	packed-bed heat-transfer area in cell j	m^2
A_{spec}	specific surface area $6(1 - \varepsilon)/D_p$	m^2/m^3
$\dot{m}_{a,j}$	air mass flow through cell j	kg/h
$C_{a,j}$	air capacity rate $\dot{m}_{a,j} c_{p,a}$	kJ/h/K
$\dot{m}_{\text{flow},j}$	clinker mass flux leaving cell j ($= m_{c,j}/\tau_j$)	kg/h
τ_j	local residence time (per cell) $= L_{\text{ch}}/v_j$	h
$Q_{\text{conv},j}$	convective heat rate clinker $\rightarrow$ air	kW
$Q_{\text{rad},j}$	net radiative heat rate from clinker	kW
$Q_{\text{amb},j}$	shell-loss heat rate per cell	kW
$Q_{\text{kiln-rad}}$	radiation from kiln end into hood	kW
$\phi_{\text{conv}}(\dot{V})$	airflow-dependent forced-convection factor $\in [0.05, 1]$	—
ε_w	NTU-effectiveness for wall w	—
$T_{\text{sec,raw}}, T_{\text{tert,raw}}, T_{\text{exh,raw}}$	flow-weighted raw air temperatures per zone	$^{\circ}\text{C}$
$T_{\text{tert,aH}}$	tertiary-air temp after hood NTU filter	$^{\circ}\text{C}$
$T_{\text{3air}}, T_{\text{xair}}$	predicted outputs (match <code>g_3air_gil_temp</code> , <code>g_xair_temp</code>)	$^{\circ}\text{C}$

Fixed physical constants & geometry

Symbol	Meaning	Default
N	number of cells (7 chambers $\times$ sub_chambers)	21
L_{ch}	cell length = grate_length / N	derived
A_{ch}	cell cross-sectional area (grate width $\times$ cell length)	fitted
$h_{\text{min}}, h_{\text{max}}$	clipping bounds on bed height	0.05, fitted
D_p	characteristic particle diameter	0.025 m
ε	bed porosity	0.4
ρ_{bulk}	bulk clinker density	1500 kg/m^3
ρ_a	hot-air density	1.1 kg/m^3
$c_{p,a}$	air specific heat	1.05 $\text{kJ}/(\text{kg}\cdot\text{K})$
$c_{p,c}$	clinker specific heat	fitted (≈ 0.63)
$\varepsilon_c, \varepsilon_a$	clinker / air emissivities (radiation term)	0.9 / 0.1
$\varepsilon_k, \varepsilon_h$	kiln / hood emissivities	0.85 / 0.8
$\mathcal{S}, \mathcal{T}, \mathcal{E}$	secondary / tertiary / exhaust chamber index sets	{1,2} / {1,2} / {3..7}

Fitted scalar parameters (the 17 degrees of freedom)

Symbol	Meaning	Param name
h_{eff}	effective convective heat-transfer coefficient	<code>convective_scale</code>
$b_j \in \mathbb{R}^7$	hot-chamber NTU boosts	<code>chamber_ntu_boost</code>
k_{amb}	linearised shell-loss constant	<code>ambient_k</code>
τ_{res}	fallback residence time	<code>residence_time_mins</code>
τ_{lag}	pure-shift throughput lag	<code>throughput_lag_mins</code>
τ_{disp}	dispersion time constant	<code>throughput_dispersion_mins</code>
γ	bypass fraction	<code>air_bypass_fraction</code>
k_v	grate-speed-to-velocity coefficient	<code>speed_coeff</code>

Symbol	Meaning	Param name
VFA	effective kiln → hood view-factor × area	<code>view_factor_area</code>
$M_w, (hA)_w, (UA)_w$	wall thermal mass and heat-transfer products	<code>{hood,tert_duct,exh_duct}_{thermal_mass,h_A,ambient_loss}</code>
$d_{\min}, d_{\max}$	damper-to-flow mapping bounds	<code>damper_{min,max}_frac</code>

2.2 State vector

The simulator carries **2N + 3** ODE states (plus two quasi-steady algebraic outputs for $T_{a,j}$):

$$\mathbf{Y}(t) = \left[T_{c,1}, \dots, T_{c,N}, m_{c,1}, \dots, m_{c,N}, T_{\text{hood}}, T_{\text{tert}}, T_{\text{exh}} \right]$$

Exogenous driver:

$$u(t) = \left[T_{\text{in}}(t), \dot{m}_{\text{in}}(t), \{\dot{V}_{a,j}(t)\}_{j=1}^N, v_1(t), v_2(t), d(t), T_{\text{kiln}}(t) \right]$$

2.3 Per-cell geometry & capacity rates

Derived per cell from its current clinker mass $m_{c,j}$:

$$h_{\text{bed},j} = \text{clip} \left(\frac{m_{c,j}}{\rho_{\text{bulk}} A_{\text{ch}}}, h_{\min}, h_{\max} \right)$$

$$V_{\text{bed},j} = A_{\text{ch}} h_{\text{bed},j}, \quad A_{\text{bed},j} = \frac{6(1-\varepsilon)}{D_p} V_{\text{bed},j}$$

$$\dot{m}_{a,j} = \dot{V}_{a,j} \rho_a \text{ [kg/h]}, \quad C_{a,j} = \dot{m}_{a,j} c_{p,a} \text{ [kJ/h/K]}$$

2.4 Gas-side energy balance (quasi-steady)

Air transit time through ~0.15 m of bed at 1–2 m/s is $\ll 1$ s, so the gas phase is solved algebraically each step. Heat pulled out of the clinker is capped by the air capacity rate (pinch-point constraint, $T_{a,\text{out}} \leq T_c$):

$$Q_{\text{conv},j} = \min \left(h_{\text{eff}} b_j A_{\text{bed},j} (T_{c,j} - T_{\text{amb}}), C_{a,j} (T_{c,j} - T_{\text{amb}}) \right)$$

$$T_{a,j} = T_{\text{amb}} + \frac{Q_{\text{conv},j}}{C_{a,j}}, \quad T_{a,j} \leq T_{c,j}$$

where b_j is the per-chamber NTU-boost (applied in hot chambers 1–2 where effective h is larger than bulk packed-bed correlations give).

2.5 Clinker-side ODE

Each clinker cell has advection in from the upstream cell, advection out downstream, convective loss to the air stream, net radiation, and a linearised shell loss. With $\tau_j = L_{\text{ch}}/v_j$ and per-cell thermal mass $M_{c,j} = m_{c,j} c_{p,c}$:

$$M_{c,j} \frac{dT_{c,j}}{dt} = \dot{m}_{\text{flow},j} c_{p,c} (T_{c,j-1} - T_{c,j}) - Q_{\text{conv},j} - Q_{\text{rad},j} - Q_{\text{amb},j}$$

$$Q_{\text{rad},j} = \sigma A_{\text{bed},j} (\varepsilon_c (T_{c,j} + 273)^4 - \varepsilon_a (T_{a,j} + 273)^4)$$

$$Q_{\text{amb},j} = M_{c,j} k_{\text{amb}} (T_{c,j} - T_{\text{amb}})$$

$$\dot{m}_{\text{flow},j} = m_{c,j-1}/\tau_{j-1}, \quad T_{c,0} \equiv T_{\text{in}}(t)$$

2.6 Clinker mass ODE (inventory tracking, COOL-04)

$$\frac{dm_{c,j}}{dt} = \frac{m_{c,j-1}}{\tau_{j-1}} - \frac{m_{c,j}}{\tau_j}, \quad m_{c,0} \equiv \dot{m}_{\text{in}}(t) \cdot \tau_1$$

Why does mass vary between chambers and over time?

- *Throughput changes.* $\dot{m}_{\text{in}}$ swings $0 \rightarrow 130$ t/h across a startup; in-chamber mass is the time-integral of inflow – outflow, so it follows a travelling wavefront.
- *Asymmetric filling.* Chamber 1 fills ~2 min after feed resumes; chamber 7 catches up ~30 min later.
- *Grate-speed changes.* τ_j shortens as grate speed rises, flushing mass through faster.
- *Residual bed at shutdown.* A thin layer remains on the grate (≈ 10 kg/cell floor) even at zero throughput.

Dispersive throughput (COOL-05): pure time-shift then a first-order low-pass:

$$\dot{m}_{\text{in,lag}}(t) = \dot{m}_{\text{in}}(t - \tau_{\text{lag}}), \quad \tau_{\text{disp}} \frac{d\tilde{\dot{m}}_{\text{in}}}{dt} = \dot{m}_{\text{in,lag}}(t) - \tilde{\dot{m}}_{\text{in}}(t)$$

2.7 Hood, tertiary-duct and exhaust-duct walls (COOL-02, COOL-03)

Three wall tanks driven by kiln radiation (hood only), forced convection from the air stream, and shell loss to ambient. Forced convection is scaled by $\phi_{\text{conv}}(\dot{V}) \in [0.05, 1]$ so convection collapses when fans are off.

$$M_{\text{hood}} \frac{dT_{\text{hood}}}{dt} = \sigma \text{VFA} \varepsilon_k \varepsilon_h ((T_{\text{kiln}} + 273)^4 - (T_{\text{hood}} + 273)^4) + (hA)_{\text{hood}} \phi_{\text{conv}}(\dot{V}_{\text{tert}}) (T_{\text{tert,raw}} - T_{\text{hood}}) - (UA)_{\text{hood}} (T_{\text{hood}} - T_{\text{amb}})$$

$$M_{\text{tert}} \frac{dT_{\text{tert}}}{dt} = (hA)_{\text{tert}} \phi_{\text{conv}}(\dot{V}_{\text{tert}}) (T_{\text{tert,aH}} - T_{\text{tert}}) - (UA)_{\text{tert}} (T_{\text{tert}} - T_{\text{amb}})$$

$$M_{\text{exh}} \frac{dT_{\text{exh}}}{dt} = (hA)_{\text{exh}} \phi_{\text{conv}}(\dot{V}_{\text{exh}}) (T_{\text{exh,raw}} - T_{\text{exh}}) - (UA)_{\text{exh}} (T_{\text{exh}} - T_{\text{amb}})$$

The air stream exchanges with each wall using an NTU-effectiveness filter — this is what produces the Phase-1 rise (cold air warmed by hot duct walls):

$$\varepsilon_w = 1 - \exp\left(-\frac{(hA)_w \phi_{\text{conv}}}{\dot{m}_a c_{p,a}}\right), \quad T_{\text{out}} = T_{\text{in}} + \varepsilon_w (T_w - T_{\text{in}})$$

2.8 Air-routing algebra (output map)

Three flow-weighted averages over disjoint-ish chamber subsets produce the measured signals. $d(t)$ is the tertiary damper position, linearly interpolated between d_{min} and d_{max} :

$$T_{\text{tert,raw}} = \frac{\sum_{j \in \mathcal{T}} \dot{V}_{a,j} T_{a,j}}{\sum_{j \in \mathcal{T}} \dot{V}_{a,j}}, \quad T_{\text{exh,raw}} = \frac{\sum_{j \in \mathcal{E}} \dot{V}_{a,j} T_{a,j}}{\sum_{j \in \mathcal{E}} \dot{V}_{a,j}}$$

$$T_{\text{3air}} = \text{NTU}_{\text{tert}}(\text{NTU}_{\text{hood}}(T_{\text{tert,raw}}, T_{\text{hood}}), T_{\text{tert}}), \quad T_{\text{xair}} = \text{NTU}_{\text{exh}}(T_{\text{exh,raw}}, T_{\text{exh}})$$

So the tertiary air sees the hood wall *and then* the duct wall; the exhaust sees only the exhaust duct.

2.9 One consolidated ODE

The whole thing is one coupled ODE $\dot{\mathbf{Y}} = f(\mathbf{Y}, u(t))$ of dimension $2N + 3 = 45$ (with $N = 21$):

$$\dot{\mathbf{Y}} = \begin{bmatrix} \dot{T}_{c,j} \\ \dot{m}_{c,j} \\ \dot{T}_{\text{hood}} \\ \dot{T}_{\text{tert}} \\ \dot{T}_{\text{exh}} \end{bmatrix} = \begin{bmatrix} M_{c,j}^{-1} (\dot{m}_{\text{flow},j} c_{p,c} \frac{\Delta T_{c,j} - Q_{\text{conv},j} - Q_{\text{rad},j} - Q_{\text{amb},j}}{\frac{\tau_{j-1}}{m_{c,j-1}} - \frac{\tau_j}{m_{c,j}}}) \\ M_{\text{hood}}^{-1} (Q_{\text{kiln-rad}} + (hA)_{\text{hood}} \phi_c (T_{\text{tert,raw}} - T_{\text{hood}}) - (UA)_{\text{hood}} (T_{\text{hood}} - T_{\text{amb}})) \\ M_{\text{tert}}^{-1} ((hA)_{\text{tert}} \phi_c (T_{\text{tert,aH}} - T_{\text{tert}}) - (UA)_{\text{tert}} (T_{\text{tert}} - T_{\text{amb}})) \\ M_{\text{exh}}^{-1} ((hA)_{\text{exh}} \phi_c (T_{\text{exh,raw}} - T_{\text{exh}}) - (UA)_{\text{exh}} (T_{\text{exh}} - T_{\text{amb}})) \end{bmatrix}$$

Strictly speaking the implementation is *semi-implicit*: clinker transport is handled as a fractional mixing step (Lie-splitting of the advection operator) and air temperatures are solved algebraically rather than by including an air ODE.

Both are standard accelerations for a 1-D plug-flow reactor — they keep the system stiff-but-not-too-stiff with explicit Euler at $\Delta t = 60$ s.

2.10 Code layout — classes and methods

All simulator code lives in `exp/users/michael/physics_simulator/cooler/cooler_model.py`:

Role	Class / function	Line	Responsibility
Main driver	<code>CoolerSimulator</code>	844	Holds a <code>CoolerSimulationConfig</code> , instantiates modules, exposes <code>simulate(...)</code> .
Config root	<code>CoolerSimulationConfig</code>	778	Pydantic tree: dt, N-chambers, sub_chambers, throughput lag/dispersion, fan model, per-module params, ablation flags.
State types	<code>CoolerTemperatures</code>	33	Per-cell clinker temps.
	<code>CoolerSimulationState</code>	69	Bundles temps + air + time — input to <code>module.compute</code> .
	<code>CoolerSimulationInputs</code>	79	Per-step exogenous drivers.
	<code>CoolerHeatFlowContribution</code>	39	dT/dt per cell + total Q; implements <code>+</code> / <code>-</code> so modules stack.
Module protocol	<code>CoolerHeatFlowModule</code>	95	Every heat-flow module implements <code>compute(state, inputs) → CoolerHeatFlowContribution</code> .
Heat-flow	<code>ClinkerTransportModule</code>	226	§2.5 advection mixing step; <code>apply_transport(...)</code> outside the summation.
	<code>ConvectiveHeatTransferModule</code>	295	§2.4 gas-side solve; <code>compute_air_out_temps(...)</code> .
	<code>RadiativeHeatTransferModule</code>	520	$\sigma \cdot A \cdot (T^4 - T_a^4)$ over the bed.
	<code>CoolerAmbientLossModule</code>	591	Linearised shell loss.
Param blocks	<code>ClinkerTransportParams</code> / <code>ConvectiveHeatTransferParams</code> / <code>RadiativeHeatTransferParams</code> / <code>AmbientLossParams</code> / <code>AirRoutingParams</code> / <code>KilnRadiationParams</code> / <code>HoodDuctParams</code> / <code>FanModelParams</code>	113–750	Pydantic parameter blocks.
Post-process	<code>compute_air_routing(...)</code>	695	§2.8 flow-weighted averages + damper.
	<code>fan_speed_to_airflow(...)</code>	766	Fan curve $\dot{V} = k \cdot \text{speed}^n$.

`CoolerSimulator` public methods:

- `__init__(config)` — build from config.
- `_init_modules()` — instantiate heat-flow modules.
- `get_enabled_modules()` — debug helper.
- `simulate(initial_clinker_temps, clinker_inlet_temps, throughputs, chamber_airflows, grate_speeds, grate2_speeds, damper_3air, n_steps, fan_speeds=None, kiln_radiation_temps=None)` — integration loop. Returns a dict with keys `t_secondary_air`, `t_tertiary_air`, `t_exhaust_air`, `t_hood`, `t_tert_duct`, `t_exh_duct`, per-chamber `t_clinker_{1..7}`, `t_air_{1..7}`, plus sub-cell resolution `t_clinker_sub_{1..21}`.

Per-step flow inside `simulate`:

1. Shift + low-pass the raw throughput to get $\tilde{m}_{\text{in}}(t)$.
2. Expand physical-chamber params to N sub-cells.

3. For each step $i = 1 \dots n$:
 - a. Update clinker mass $m_{c,j}$ (advection + inlet rate + residual floor).
 - b. `ClinkerTransportModule.apply_transport` → new clinker temps.
 - c. Sum `compute()` over convective / radiative / ambient → total dT/dt .
 - d. Explicit-Euler integrate, clip to $[T_{\text{amb}}, 1500 \text{ °C}]$.
 - e. Re-solve quasi-steady $T_{a,j}$ via `compute_air_out_temps`.
 - f. `compute_air_routing` → $T_{\text{sec,raw}}, T_{\text{tert,raw}}, T_{\text{exh,raw}}$.
 - g. Integrate wall ODEs and NTU-filter air streams to get $T_{\text{3air}}, T_{\text{xair}}$.
4. Aggregate sub-cells back to 7 physical chambers and return.

Known display issue: `t_secondary_air`, `t_tertiary_air`, `t_exhaust_air` are allocated with `np.zeros(n_steps)` and the loop starts at `i = 1`, so index 0 stays at 0 °C — visible as a spurious zero at the left edge of every plot.

3. How the model is evaluated

3.1 Steady-state harness — `plot_rollout.py`

Loads a 24 h window (2024-10-01) of `mapped` MOK data via `load_plant_data`, estimates per-chamber airflows from `g_c_gol_airflow` apportioned by fan speed, rolls out the simulator at 1-minute resolution, and scores MAE and bias for the two fit targets plus the two validation targets.

3.2 Startup harness — `evaluate_startup.py` & `find_startup.py`

Scans MOK data for shutdown → startup transitions by finding the longest contiguous zero-throughput run. Oct 10 2024 (515-min shutdown) selected as the canonical case. The window extends 60 min *before* the ramp (to capture the Phase 1 rise from kiln radiation / duct thermal mass) and ends once throughput stabilises.

Three-phase behaviour targeted:

- **Phase 1** — slow rise to ~200–350 °C from hot hood / kiln radiation before clinker arrives.
- **Phase 2** — dip as ID fans pull cold ambient through the just-opened dampers.
- **Phase 3** — steep rise to ~600–700 °C about 30 min after throughput ramps (lag + transport).

3.3 Fitting — `fit_params.py`

SciPy `differential_evolution` (popsize=15, maxiter=50, seed=42) over the 17 parameters of §2.1. The objective combines startup and steady-state MAE with a 2× weight on `g_3air_gil_temp`:

$$J(\theta) = 2 \text{MAE}_{\text{3air}}^{\text{startup}}(\theta) + \text{MAE}_{\text{xair}}^{\text{startup}}(\theta) + 2 \text{MAE}_{\text{3air}}^{\text{ss}}(\theta) + \text{MAE}_{\text{xair}}^{\text{ss}}(\theta)$$

What is Differential Evolution?

A population-based, gradient-free global optimiser for real-valued, box-constrained problems (Storn & Price, 1997). It evolves a population of candidate parameter vectors by mixing them using vector differences, keeping each new vector only if it improves the objective.

With population $\{\theta_1, \dots, \theta_P\} \subset \mathbb{R}^{17}$:

1. **Mutate** — for each target θ_i , sample three distinct members a, b, c and form donor $\mathbf{v} = \theta_a + F(\theta_b - \theta_c)$ (here $F \approx 0.5$).
2. **Crossover** — build trial $\mathbf{u}$ by copying each coordinate from $\mathbf{v}$ with probability CR , else from θ_i .
3. **Select** — keep $\mathbf{u}$ if $J(\mathbf{u}) < J(\theta_i)$.
4. Repeat until convergence (`tol`) or `maxiter`.

With `popsizer=15, maxiter=50` SciPy uses $P = 15 \cdot 17 = 255$ vectors and up to 50 generations $\approx 13,000$ evaluations; in practice converged after ~ 750 .

Why DE (and not something else)?

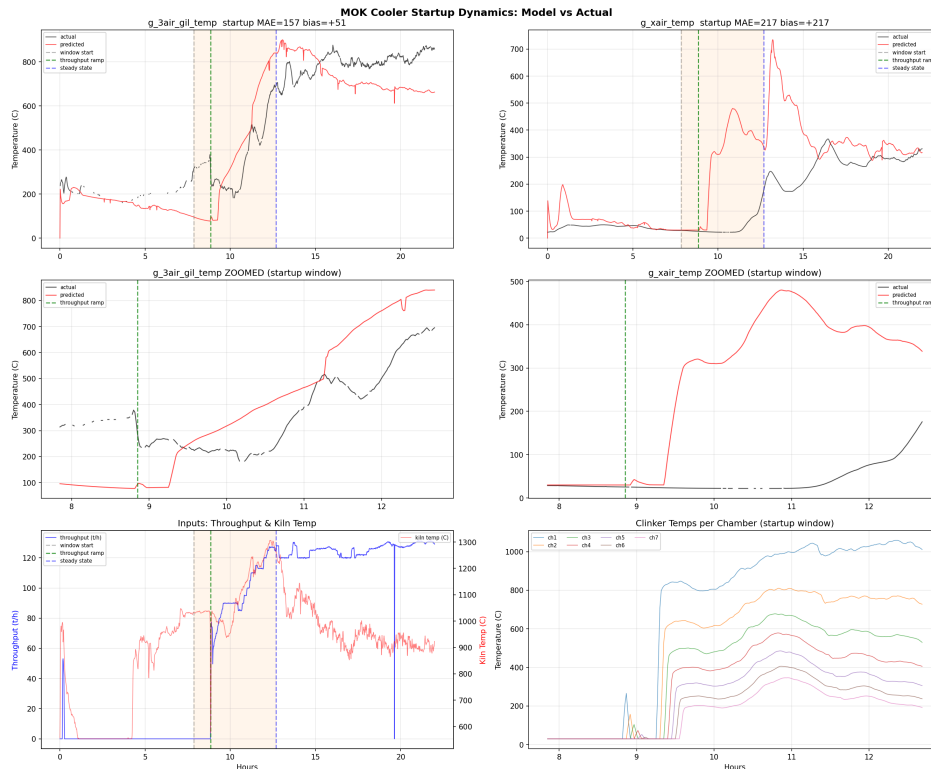
- **Non-smooth objective.** The rollout contains `min()` caps (§2.4), clipping, an integer-shift throughput lag, and piecewise-linear damper modulation. Gradient methods stall.
- **No analytic gradient.** Numerical gradient would require 17 finite differences per step (~ 34 rollouts); DE needs only function values.
- **Multi-modal.** Parameters trade off (e.g. M_{hood} vs $(hA)_{\text{hood}}$). A single-basin optimiser latches onto whichever minimum is nearest the start.
- **Boxed bounds, no equality constraints.** Exactly what DE is designed for.
- **17 dimensions is a sweet spot.** DE scales linearly with dimension; at 17 it converges in minutes.

Alternatives worth trying: **CMA-ES** handles correlated parameters more gracefully; **Bayesian optimisation** shines when evaluations are expensive (ours aren't); **gradient-based fitting on a smoothed surrogate** (soft-min, soft-clip) becomes attractive for $< 1\%$ MAE refinement.

4. Current performance

4.1 Oct 10 2024 — startup window

Both fit targets tracked over a 515-min shutdown followed by a full startup. Phase 1/2/3 dynamics reproduced qualitatively.



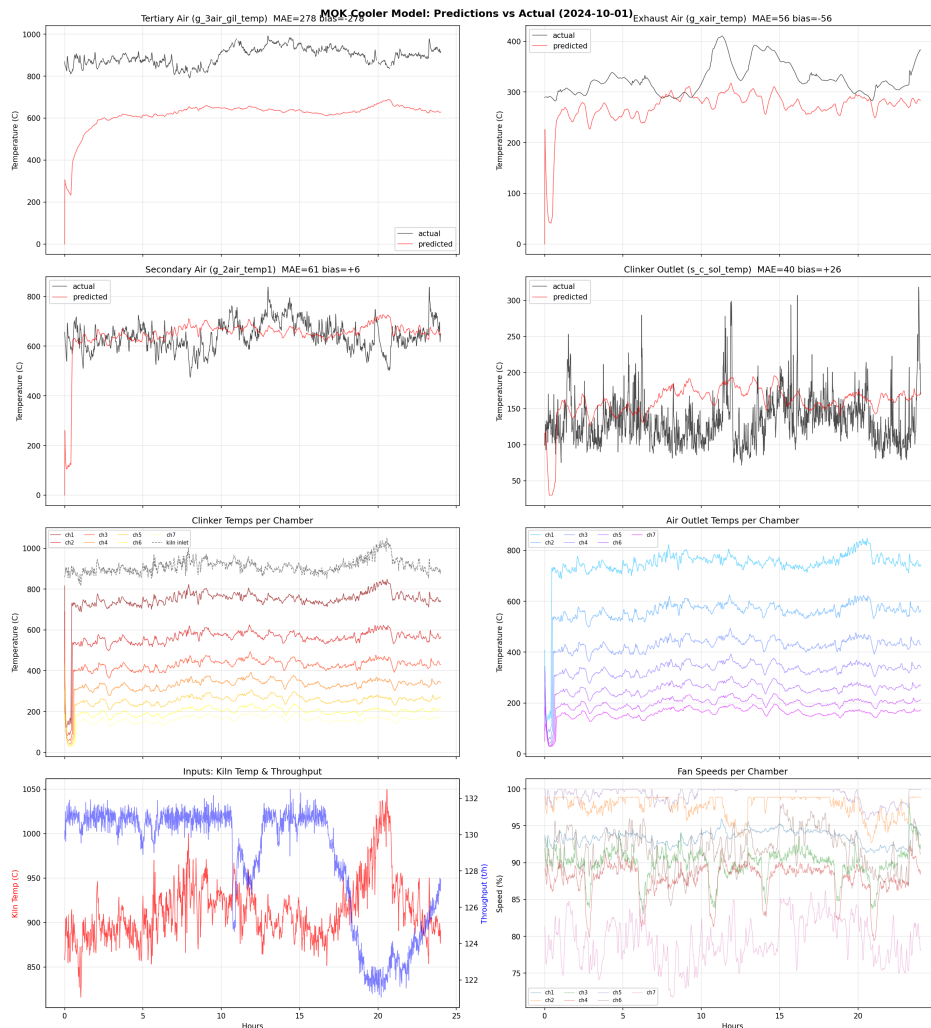
Target	Startup MAE (°C)	Startup bias (°C)	Full-window MAE (°C)	Notes
<code>g_3air_gil_temp</code>	85 / 157	+1 / +51	117	Fit target. 85 = DE's score (sub_chambers=1); 157 = <code>evaluate_startup</code> rollout at sub_chambers=3.

Target	Startup MAE (°C)	Startup bias (°C)	Full-window MAE (°C)	Notes
g_xair_temp	122 / 217	+122 / +217	105	Fit target. Systematic positive bias — model runs warmer than sensor.
g_2air_temp1	291 / 426	+284 / +417	162	Validation only. Secondary air tracks well in steady state but misses startup draw-down.
s_c_sol_temp	63 / 137	+62 / +137	97	Validation only. Clinker outlet broadly tracks.

Pre-branch baseline: 3air MAE=165, xair MAE=21, combined=186. The branch drives 3air down at the expense of xair warming up — Oct-10 xair sensor reads ~40 °C for the whole window, which is suspiciously low; the model cannot physically produce that given the measured airflows.

4.2 Oct 1 2024 — steady-state rollout

A plain 24 h window with no deep shutdown, sanity check for the DE-fit parameters.



Target	MAE (°C)	Bias (°C)	Notes
g_3air_gil_temp	278	-278	Under-predicts: fit dominated by startup transient; steady-state gain compressed by high hood_h_A.
g_xair_temp	57	-56	Good.
g_2air_temp1	61	+6	Good — validation signal well predicted without being fit.

Target	MAE (°C)	Bias (°C)	Notes
s_c_sol_temp	40	+26	Good.

5. Fitted parameter values

Current best from the latest DE run (seed=42, 17 params, objective = 206). Bold values are at or within 5% of their bound.

Parameter	Value	Bounds	Role
residence_time_mins	88.1	[5, 120]	Thermal-mass fallback.
convective_scale (h_{eff})	57.4 W/m ² K	[0.1, 100]	Base convective h.
speed_coeff	0.00269	[1e-5, 5e-3]	Grate-speed → velocity scale.
ntu_boost_hot	11.9	[1, 30]	Hot-chamber h multiplier.
c_p_clinker	0.63 kJ/kgK	[0.5, 2.0]	Clinker specific heat.
damper_min_frac	0.32	[0.3, 1.0]	At lower bound.
chamber_area	8.1 m ²	[1, 20]	Grate width × chamber length.
bed_height_max	0.14 m	[0.1, 1.5]	Near lower bound.
grate_length	33.3 m	[10, 50]	
ambient_k	0.059	[0.01, 1.0]	Shell-loss rate.
view_factor_area	1.5 m ²	[1, 30]	At lower bound.
hood_thermal_mass	9919 kJ/K	[500, 10000]	At upper bound.
hood_h_A	4.92 kW/K	[0.05, 5.0]	At upper bound.
hood_ambient_loss	0.055	[0.05, 3.0]	At lower bound.
tert_duct_thermal_mass	4595 kJ/K	[200, 5000]	Near upper bound.
tert_duct_h_A	2.93 kW/K	[0.05, 3.0]	At upper bound.
air_bypass_fraction	0.45	[0.0, 0.9]	Fraction of air short-circuiting the bed.

6. Open issues and candidate next steps

- **Five of 17 parameters are on their bounds.** Widen bounds on `hood_thermal_mass`, `hood_h_A`, `tert_duct_thermal_mass`, `tert_duct_h_A`, `damper_min_frac`, and refit — or accept that the optimiser is compensating for a missing physical term (e.g. direct chamber-1 clinker radiation to the hood wall, currently lumped into the kiln-radiation term).
- **xair bias is persistent.** Oct 10 `g_xair_temp` hovers near ambient (~40 °C) while measured exhaust airflows are substantial. The sensor may be on a cold duct leg, or the flow split is wrong — worth verifying with a second startup case before further tuning.
- **Fit single-condition.** Only Oct 10 has been used for fitting. `find_startup.py` identified 11 qualifying events in Sep–Nov 2024; a multi-window fit would reduce overfitting to Oct 10's specific damper schedule.
- **Steady-state 3air gap.** Oct 1 `g_3air_gil_temp` MAE = 278 °C suggests the startup-weighted objective pulled the damper model into an unphysical regime for normal running.
- **(Michael) Fit over steady state data**
- `evaluate_startup.py::get_best_config` and `plot_rollout.py::best_config` are duplicate best-param definitions — consolidate into a single source of truth before the next fit.
- Fix the index-0 initialisation issue in `simulate(...)` so plots don't start at 0 °C.